

EXPERT MACHINE LEARNING MODULE

KDD Cup 1999 Autoencoder Anomaly Detection

Clear workflow summary with actual variables, methods, outputs, and interpretation

Prepared by **Ruturaj Mokashi**

Created on 3 June 2026

What this PDF gives you

A start-to-finish explanation of the KDD Autoencoder project. Every step explains what happened, why it was needed, which variables were used, and what output was created.

Executive Overview

This project moved from raw KDD Cup 1999 network connection data to a trained Autoencoder anomaly detection model. The model learned normal traffic patterns, calculated reconstruction error on test connections, and used a threshold to flag likely attacks.

Core idea

Train on normal rows only. If a future connection cannot be reconstructed well, treat it as suspicious.

Main Workflow Map



Final Result

Model	Precision	Recall	F1-score	Accuracy	Missed attacks
Autoencoder	0.9879	0.9984	0.9931	0.9889	125
Isolation Forest	0.9886	0.9903	0.9894	0.9830	773

Final interpretation

The Autoencoder won because it achieved the higher F1-score and recall, and it missed far fewer attacks. In network security, missed attacks are usually more dangerous than false alarms.

Step 1 - Load the KDD dataset

What we did	Why we did it
Loaded the KDD Cup 1999 10% dataset using <code>fetch_kddcup99()</code> .	We needed the raw network connection features and their original labels before any cleaning or modeling.
Variables / methods used	Output created
<code>fetch_kddcup99, RANDOM_STATE, kdd_data</code>	<code>X_kdd_raw</code> : raw feature dataframe with 41 columns. <code>y_kdd_raw</code> : raw target labels.
<pre>kdd_data = fetch_kddcup99(percent10=True, as_frame=True, shuffle=True, random_state=RANDOM_STATE) X_kdd_raw = kdd_data.data.copy() y_kdd_raw = kdd_data.target.copy()</pre>	
Result / interpretation Dataset loaded successfully: 494,021 rows, 41 feature columns, and 494,021 target labels.	

Step 2 - Inspect original labels

What we did	Why we did it
Converted labels to string format and counted how many rows each label had.	The original target was not just normal vs anomaly. It contained many attack names such as smurf, neptune, back, and satan.
Variables / methods used	Output created
<code>y_kdd_raw, astype(str), value_counts()</code>	<code>y_kdd</code> : readable string labels. <code>label_counts</code> : count and percentage table for original labels.
<pre>y_kdd = y_kdd_raw.astype(str) label_counts = y_kdd.value_counts().reset_index()</pre>	
Result / interpretation Found 23 unique labels. The largest labels were smurf, neptune, and normal. This confirmed the dataset had one normal class and many attack classes.	

Step 3 - Convert labels into normal vs anomaly

What we did	Why we did it
Cleaned the label text and converted the target into binary labels: 0 for normal, 1 for anomaly.	Autoencoder evaluation needs a clear target. We do not need every attack type for this exercise; we need normal vs attack/anomaly.
Variables / methods used	Output created
<code>y_kdd, y_kdd_clean, np.where()</code>	<code>y_kdd_binary</code> : binary target vector. 0 = Normal, 1 = Anomaly.

```
y_kdd_clean = (
    y_kdd.str.replace("b'", "", regex=False)
    .str.replace("'", "", regex=False)
    .str.replace(".", "", regex=False)
)

y_kdd_binary = np.where(y_kdd_clean == "normal", 0, 1)
```

Result / interpretation

Normal: 97,278 rows (19.69%). Anomaly: 396,743 rows (80.31%). The dataset is attack-heavy.

Step 4 - Correct feature types

What we did	Why we did it
Fixed the feature columns because many numeric values were loaded as text.	Neural networks and preprocessing pipelines need correct feature types. We needed to know what to scale and what to one-hot encode.
Variables / methods used	Output created
X_kdd_raw, pd.to_numeric(), manual categorical list	X_kdd_clean: corrected dataframe. corrected_numerical_columns: 38 columns. corrected_categorical_columns: 3 columns.

```
X_kdd_clean = X_kdd_raw.copy()

categorical_columns = ["protocol_type", "service", "flag"]
numerical_columns = [col for col in X_kdd_clean.columns
                      if col not in categorical_columns]

for col in numerical_columns:
    X_kdd_clean[col] = pd.to_numeric(X_kdd_clean[col], errors="coerce")

corrected_numerical_columns = X_kdd_clean.select_dtypes(include=["number"]).columns.tolist()
corrected_categorical_columns = categorical_columns
```

Result / interpretation

Corrected feature structure: 38 numerical columns and 3 categorical columns: protocol_type, service, flag.

Step 5 - Create train/test split and normal-only training data

What we did	Why we did it
Split the dataset into training and testing sets, then extracted only normal rows from training data.	The Autoencoder should learn normal traffic first. Therefore, it should be trained only on normal training rows.
Variables / methods used	Output created
train_test_split(), X_kdd_clean, y_kdd_binary	X_train_raw, X_test_raw, y_train, y_test, X_train_normal_raw

```
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X_kdd_clean,
    y_kdd_binary,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y_kdd_binary
)

X_train_normal_raw = X_train_raw[y_train == 0].copy()
```

Result / interpretation

Training set: 395,216 rows. Test set: 98,805 rows. Normal rows for Autoencoder training: 77,822 rows.

Step 6 - Preprocess features

What we did	Why we did it
Scaled numerical columns and one-hot encoded categorical columns using a ColumnTransformer pipeline.	The Autoencoder needs all input values as numeric features. Raw categories like protocol_type and service cannot be sent directly into a neural network.
Variables / methods used	Output created
ColumnTransformer, Pipeline, SimpleImputer, StandardScaler, OneHotEncoder	X_train_normal_processed, X_train_processed, X_test_processed

```

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("numeric", numeric_pipeline, corrected_numerical_columns),
    ("categorical", categorical_pipeline, corrected_categorical_columns)
])

preprocessor.fit(X_train_normal_raw)

X_train_normal_processed = preprocessor.transform(X_train_normal_raw).astype("float32")
X_train_processed = preprocessor.transform(X_train_raw).astype("float32")
X_test_processed = preprocessor.transform(X_test_raw).astype("float32")

```

Result / interpretation

The original 41 features became 74 processed numerical features. Autoencoder training input shape: 77,822 x 74. Test input shape: 98,805 x 74.

Step 7 - Build the Autoencoder

What we did	Why we did it
Created a Keras Sequential Autoencoder with encoder, bottleneck, decoder, and output layers.	The Autoencoder must compress normal traffic into a smaller representation, then reconstruct it. Poor reconstruction later indicates possible anomaly.
Variables / methods used	Output created
keras.Sequential, layers.Dense, input_dim	autoencoder: compiled neural network model.

```
input_dim = X_train_normal_processed.shape[1]

autoencoder = keras.Sequential([
    layers.Input(shape=(input_dim,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(input_dim, activation="linear")
])

autoencoder.compile(optimizer="adam", loss="mse")
```

Result / interpretation

Architecture: 74 -> 64 -> 32 -> 16 -> 32 -> 64 -> 74. The bottleneck has 16 neurons. Total trainable parameters: 14,874.

Step 8 - Train the Autoencoder

What we did	Why we did it
Trained the Autoencoder using normal processed rows as both input and target.	An Autoencoder learns by reconstructing its own input. Training on normal rows teaches it what normal traffic looks like.
Variables / methods used	Output created
<code>autoencoder.fit(), X_train_normal_processed, history</code>	history: training and validation loss per epoch.

```
history = autoencoder.fit(
    X_train_normal_processed,
    X_train_normal_processed,
    epochs=20,
    batch_size=256,
    validation_split=0.20,
    shuffle=True
)
```

Result / interpretation

Final training loss: 0.026720. Final validation loss: 0.171016. Training loss decreased strongly, so the model learned normal traffic patterns.

Step 9 - Calculate reconstruction error on test data

What we did	Why we did it
Used the trained Autoencoder to reconstruct test rows, then calculated mean squared error for each row.	Reconstruction error becomes the anomaly score. Low error means likely normal; high error means suspicious.
Variables / methods used	Output created
<code>autoencoder.predict(), X_test_processed, np.mean(), np.square()</code>	<code>X_test_reconstructed, test_reconstruction_error, reconstruction_results</code>

```

X_test_reconstructed = autoencoder.predict(X_test_processed, verbose=0)

test_reconstruction_error = np.mean(
    np.square(X_test_processed - X_test_reconstructed),
    axis=1
)

reconstruction_results = pd.DataFrame({
    "true_label": y_test,
    "connection_type": pd.Series(y_test).map({0: "Normal", 1: "Anomaly"}),
    "reconstruction_error": test_reconstruction_error
})

```

Result / interpretation

Median error for normal rows: 0.003227. Median error for anomaly rows: 0.307520. Anomalies had much higher reconstruction error.

Step 10 - Choose threshold and predict anomalies

What we did	Why we did it
Calculated reconstruction error on normal training rows and used the 95th percentile as the anomaly threshold.	A score alone is not enough. We need a threshold to convert reconstruction error into a final normal/anomaly prediction.
Variables / methods used	Output created
X_train_normal_reconstructed, train_normal_reconstruction_error, np.percentile()	anomaly_threshold, predicted_label, predicted_type

```

X_train_normal_reconstructed = autoencoder.predict(X_train_normal_processed, verbose=0)

train_normal_reconstruction_error = np.mean(
    np.square(X_train_normal_processed - X_train_normal_reconstructed),
    axis=1
)

anomaly_threshold = np.percentile(train_normal_reconstruction_error, 95)

reconstruction_results["predicted_label"] = (
    reconstruction_results["reconstruction_error"] > anomaly_threshold
).astype(int)

```

Result / interpretation

Predicted anomaly: 80,198 rows (81.17%). Predicted normal: 18,607 rows (18.83%).

Step 11 - Evaluate the Autoencoder

What we did	Why we did it
Compared Autoencoder predictions against the true test labels.	Evaluation tells us whether the reconstruction-error approach actually detects attack-like connections well.
Variables / methods used	Output created
precision_score, recall_score, f1_score, accuracy_score, confusion_matrix	autoencoder_metrics, cm_autoencoder

```

y_true_autoencoder = reconstruction_results["true_label"].astype(int)
y_pred_autoencoder = reconstruction_results["predicted_label"].astype(int)

autoencoder_precision = precision_score(y_true_autoencoder, y_pred_autoencoder)
autoencoder_recall = recall_score(y_true_autoencoder, y_pred_autoencoder)
autoencoder_f1 = f1_score(y_true_autoencoder, y_pred_autoencoder)
autoencoder_accuracy = accuracy_score(y_true_autoencoder, y_pred_autoencoder)

cm_autoencoder = confusion_matrix(y_true_autoencoder, y_pred_autoencoder)

```

Result / interpretation

Autoencoder: precision 0.9879, recall 0.9984, F1-score 0.9931, accuracy 0.9889. Missed attacks: 125.

Step 12 - Build Isolation Forest baseline

What we did	Why we did it
Trained Isolation Forest on the same normal processed training rows and evaluated it on the same test set.	A baseline lets us compare the neural-network Autoencoder against a traditional anomaly detection method.

Variables / methods used	Output created
IsolationForest, score_samples(), np.percentile()	if_threshold, if_predicted_label, if_metrics, cm_if

```

isolation_forest_kdd = IsolationForest(
    n_estimators=100,
    contamination="auto",
    random_state=RANDOM_STATE,
    n_jobs=-1
)

isolation_forest_kdd.fit(X_train_normal_processed)

train_normal_if_scores = -isolation_forest_kdd.score_samples(X_train_normal_processed)
if_threshold = np.percentile(train_normal_if_scores, 95)

test_if_scores = -isolation_forest_kdd.score_samples(X_test_processed)

reconstruction_results["if_anomaly_score"] = test_if_scores
reconstruction_results["if_predicted_label"] = (
    reconstruction_results["if_anomaly_score"] > if_threshold
).astype(int)

```

Result / interpretation

Isolation Forest: precision 0.9886, recall 0.9903, F1-score 0.9894, accuracy 0.9830. Missed attacks: 773.

Step 13 - Compare models and select winner

What we did	Why we did it
Combined Autoencoder and Isolation Forest metrics, then compared missed attacks and final performance.	In network security, missed attacks are more dangerous than false alarms. The winner should catch more attacks while maintaining strong precision.

Variables / methods used	Output created
autoencoder_metrics, if_metrics, cm_autoencoder, cm_if, pd.concat()	model_comparison, security error comparison, final winner.

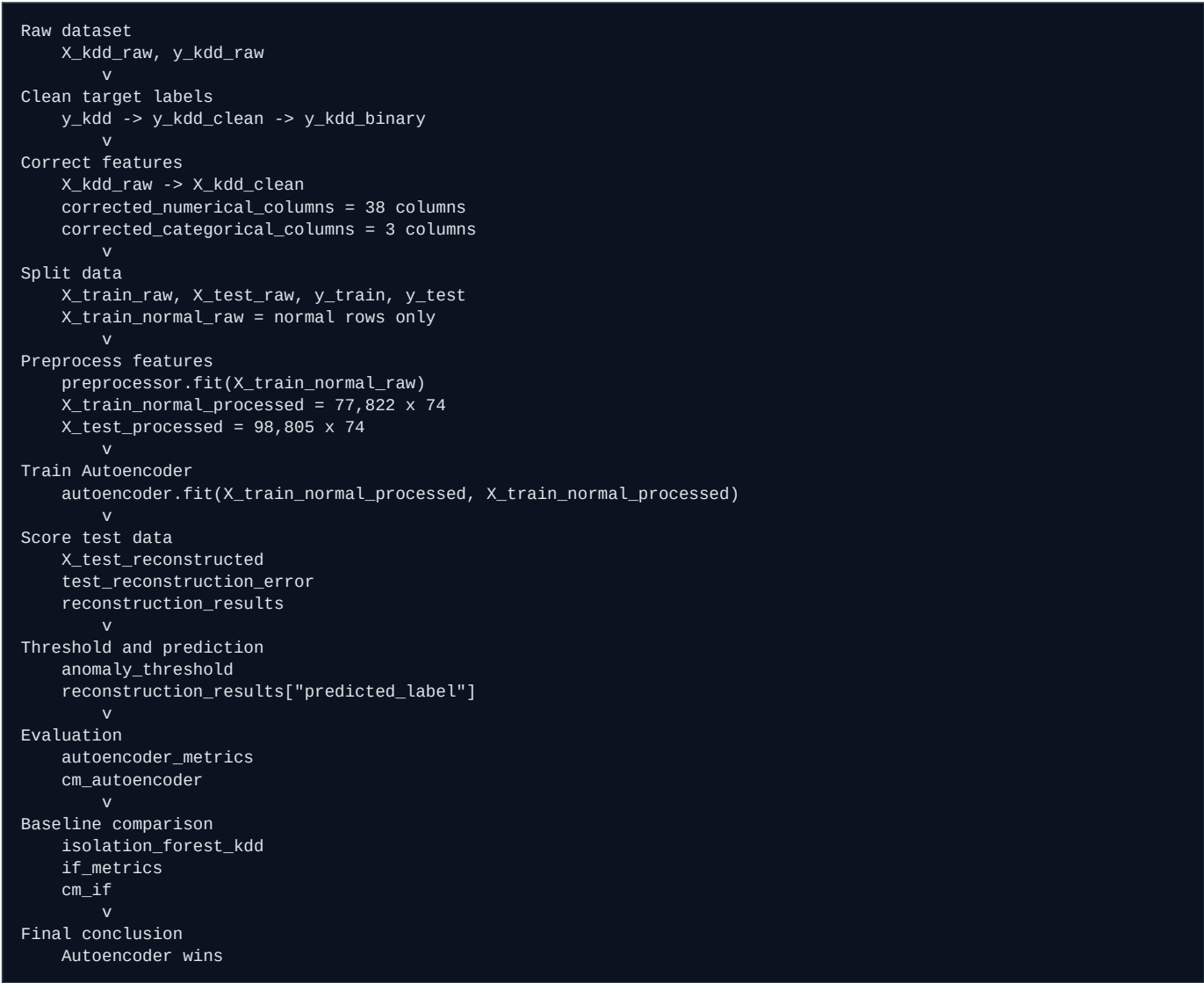

```
model_comparison = pd.concat(  
    [autoencoder_metrics, if_metrics],  
    ignore_index=True  
)  
  
model_comparison["false_alarms"] = [cm_autoencoder[0, 1], cm_if[0, 1]]  
model_comparison["missed_attacks"] = [cm_autoencoder[1, 0], cm_if[1, 0]]
```

Result / interpretation

Autoencoder won. It missed only 125 attacks, while Isolation Forest missed 773 attacks. Autoencoder also achieved the higher F1-score: 0.9931 vs 0.9894.

End-to-End Variable Flow Map

This map shows how the actual variables moved through the full project.



Final Model Comparison

Model	Precision	Recall	F1-score	Accuracy	False alarms	Missed attacks
Autoencoder	0.9879	0.9984	0.9931	0.9889	974	125
Isolation Forest	0.9886	0.9903	0.9894	0.9830	910	773

Final winner

The Autoencoder is the stronger model for this KDD Cup 1999 anomaly detection task because it has the highest F1-score and recall, and it missed far fewer attacks.

Short Interview-Style Explanation

I trained an Autoencoder on normal KDD network traffic only. After preprocessing the data into 74 numeric features, the Autoencoder learned to reconstruct normal connections. I then calculated reconstruction error on test rows and used the 95th percentile of normal training error as the anomaly threshold. The Autoencoder achieved an F1-score of 0.9931 and missed only 125 attacks. I compared it with Isolation Forest, which had an F1-score of 0.9894 but missed 773 attacks. Since missed attacks are critical in security, the Autoencoder was the better model.